Recursion type

SOURCE (HTDP, MIT 6.102)

Structural recursion LYAH ×2 · SICP · HtDP ×7 · DCIC ×2 · MIT 6.102 ×2 · LeetCode ×14
Follows the data definition: first/rest, left/right.

Generative recursion LYAH · SICP · HtDP ×2 · CLRS · LeetCode ×15
Subproblems are computed, not parts of the data.

CALLS PER STEP (SICP §1.2)

Linear recursion LYAH ×2 · SICP · HtDP ×7 · DCIC ×2 · LeetCode ×18
One recursive call per step.

Tree recursion LYAH · SICP · HtDP ×2 · CLRS · MIT 6.102 ×2 · LeetCode ×11
Two or more recursive calls per step.

PROCESS (SICP §1.2.1)

Recursive process
LYAH ×3 · SICP ×2 · HtDP ×9 · DCIC ×2 · CLRS · MIT 6.102 ×2 · LeetCode ×28
Work deferred and done on the way back.

Iterative process LeetCode
State carried down; nothing left to do on return.

Self-referential data

DATA

List HtDP · DCIC
A list whose rest is again a list.

Tree MIT 6.102 ×2
A tree whose children are trees.

Nested doll HtDP
A pictorial object holding a smaller copy.

NODE CONTENT

Concrete data DCIC · MIT 6.102
Values of the input itself. `[1, 4, 3]`

Call with arguments SICP · LeetCode ×3
The call as written, concrete args. `fib 3`

State tuple LeetCode ×3
Only the changing parameters. `(i, k)`

Partial expression SICP · HtDP · LeetCode
Half-evaluated term, as in a trace. `(* 6 (f 5))`

Base case

Marked LeetCode ×4
Labelled or highlighted as the base case.

Reached, not marked LYAH
The last call is drawn but not singled out.

Dead-end leaf LeetCode
A failed branch ends the recursion.

Decomposition

STRUCTURE

Nested containment
Calls drawn inside their caller.
- *Box-in-box* HtDP
- *Brackets* LYAH · LeetCode ×2 — `f(a(b(c)))`

Tree (node-link)
Calls linked to their subcalls.
- *Call tree* SICP · LeetCode ×3
- *State tree* LeetCode ×2
- *Choice tree* LeetCode
- *Result tree* LeetCode ×6

Linear chain
List of nodes and links.
- *Box-and-pointer* HtDP · LeetCode
- *Pointer rewiring* LeetCode ×8

Stack LeetCode ×2
Frames piled up by depth.

Table HtDP · LeetCode
Args, intermediates and results.

Substitution trace SICP · HtDP ×3 · LeetCode
Expression rewritten until a value.

Indentation staircase DCIC · LeetCode
Depth shown by indentation.

Partition LeetCode
Input split into boxed parts.

Instance mapping LeetCode ×3
Size n aligned to size n−1.

Metaphor / pictorial LYAH · HtDP · LeetCode
Dolls, mirrors, spirals.

CHOICES

Choice label LeetCode ×2
The decision named on the edge.

Prune / fail + legend LeetCode ×3
Dead branches, keyed in a legend.

ELISION (CALLS)

Full expansion SICP · LeetCode ×2
Small input, every call drawn.

Patterned elision CLRS
Rule implied by a few levels.

Unpatterned elision LeetCode
No inference can be made.

Deferred call
Subcall left unexpanded.
- *Trace term* HtDP — `f(rest…)`
- *Done box* LeetCode ×2 — `done`

Trusted subcall LeetCode ×5
The subcall's result is used without drawing inside it (leap of faith).

Return & combine

FLOW

Upward only HtDP
Results merging toward the root.

Round trip SICP · LeetCode ×7
Down to the base case, then back up.

Rewrite in place SICP · HtDP ×2
Time runs down the lines.
`f(3)` ⇒ `3·f(2)` ⇒ `3·2·f(1)` ⇒ `6`

MARKS

Return value LeetCode ×3
Value written on the way back. `= 8`

Combine step SICP · HtDP · LeetCode ×2
How sub-results merge is drawn.

Traversal path SICP
Evaluation order traced.

Call-step label LeetCode ×2
Each frame or step numbered. `call 1` `call 2`

ELISION (RETURNS)

All returns drawn SICP ×2 · HtDP ×3 · LeetCode ×7
Every call's result is shown coming back.

Some returns drawn LeetCode
Only some results, e.g. the final one.

Returns not drawn LeetCode ×4
Calls only; for a recursive process the way back is missing.

Global choices

INSTANCE

Concrete instance LYAH ×2 · SICP ×2 · HtDP ×6 · DCIC · MIT 6.102 · LeetCode ×28
Real input values.

Symbolic CLRS · LeetCode
Parametric size, as in a recurrence. `f(n/b)`

PANELS

Over time HtDP · LeetCode ×14
Snapshots of one run.

Multiple scenarios LeetCode ×4
Cases of one step side by side.

Multiple representations HtDP
Same recursion, two encodings. `(cons 1 (cons 2 '()))`

Other

Role colour LYAH · LeetCode ×2
Colour by role (pivot, done). `pivot` `done`

Cursor / pointer LeetCode ×3
Indices into the data. `i` `j`

Overlapping subproblems LeetCode ×2
The same subcall recurs.

Duplicate marker LeetCode
Repeated subproblems circled.

Cost
Work done per call or level.
- *Per level* CLRS
- *Total* CLRS · LeetCode

Code Correspondence

Diagram is code SICP · HtDP · DCIC
Trace written in the program's own syntax.

Linked to the code
Placed next to it, arrowed into it, or sharing its names.
- *Side by side* LYAH ×2 · SICP · LeetCode ×4
- *Arrows into code* HtDP · LeetCode
- *Shared identifiers* SICP · LeetCode ×3

Linked through math or prose
An equation or a written explanation, not the code.
- *Recurrence only* CLRS · LeetCode ×2
- *Prose only* LeetCode ×2

LYAH Learn You a Haskell · SICP Structure and Interpretation of Computer Programs · HtDP How to Design Programs · DCIC A Data-Centric Introduction to Computing · CLRS Introduction to Algorithms
MIT 6.102 MIT 6.102 course notes · LeetCode LeetCode solutions, vote ≥ 5 · ×n number of examples

Figure X: A design space for recursion diagrams. The recursion type is set by the problem; each recursion concept lists the ways it is drawn.